

Developing hybrid quantum-classical software: a software product line approach

Samuel Sepúlveda
Department of Computer Sciences
and Informatics, University of La
Frontera
Temuco, Chile
samuel.sepulveda@ufrontera.cl

Mario Piattini
Institute of Technologies and
Information Systems, University of
Castilla-La Mancha
Ciudad Real, Spain
mario.piattini@uclm.es

Ricardo Pérez-Castillo
Faculty of Social Sciences &
Information Technology, University
of Castilla-La Mancha
Talavera de la Reina, Spain
ricardo.pdelcastillo@uclm.es

ABSTRACT

Quantum computing is rapidly emerging as a transformative force in technology. In the near future we will increasingly encounter hybrid systems that combine quantum technology with classical software. Software engineering techniques will be needed to manage the complexity of designing such systems and their reuse. This paper introduces preliminary ideas for developing quantum-classical software using a Software Product Line approach in line with the Model-Driven Engineering principles. This approach addresses the mentioned challenges and drafts a framework for developing hybrid quantum-classical software. The preliminary insights show the feasibility and suitability of applying the proposed approach for developing complex quantum-classical software systems with high levels of variability.

CCS CONCEPTS

• **Software and its engineering** → *Software design tradeoffs*; **Software system models**;

KEYWORDS

Quantum computing, Software Product Lines, Variability, Feature Modeling

ACM Reference Format:

Samuel Sepúlveda, Mario Piattini, and Ricardo Pérez-Castillo. 2024. Developing hybrid quantum-classical software: a software product line approach. In *2024 ACM/IEEE International Workshop on Quantum Software Engineering (Q-SE 2024)*, April 16, 2024, Lisbon, Portugal. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3643667.3648224>

1 INTRODUCTION

Quantum Computing (QC) emerges not as a new traditional programming paradigm, but as a revolutionary computational architecture with an innovative approach. However, many current Software Engineering (SE) concepts and techniques designed to be platform-independent will remain relevant, although they will need adaptations for the unique characteristics of QC [15]. The same

authors posit that current quantum software development is primarily focused on low levels of abstraction, evidencing the need for engineering support at higher levels. Although such higher levels of abstraction are not yet fully developed for quantum computing, it is essential to address current challenges to pave the way for future innovations [9].

Quantum Software Engineering (QSE) aims to develop large-scale quantum software. This involves designing and coding QC-specific applications to address its unique challenges [7]. Zhao [24] describes QSE as applying software engineering principles to create and maintain quantum software, considering distinct aspects like superposition and entanglement. Despite progress in quantum software tools, a comprehensive toolkit for quantum software development is still evolving [3]. The complexity of quantum programming and algorithm creation requires formal software development techniques [14]. Consequently, Serrano et al. [20] suggest that QSE needs specialized methods and tools to address quantum-specific challenges and effectively ensure quantum systems' reliability.

Currently, "quantum applications are in most cases hybrid" [19]. The Software Engineering Institute (SEI) [9] stated a further research is needed on how to "build hybrid algorithms that leverage the best quantum and classical algorithms working together". The development lifecycle of these hybrid software systems entails several challenging factors and many decision-making points [23]. Thus, a high variability must be managed during the whole lifecycle, which imply to answer many questions. For example, should a given functionality be supported by classic software or by the quantum counterpart? What kind of quantum software should be used, gate-based or annealing? What quantum algorithm might be used for solving that functionality? What is the quantum hardware should we choose, and what are their constraints?

The Software Product Line (SPL) approach considers intensive software systems that share a standard set of features to meet the specific needs of a market segment [12]. SPLs are developed from a standard set of pre-established building blocks, including their architecture, software components, and products [8].

A key concept in SPL development is variability, which gives SPLs the flexibility and diversity needed to differentiate products [16]. It is expected to describe SPLs and manage their variability with the help of a Feature Model (FM), whose structure is a kind of tree in which its root node represents the product family, and the features are organized along the tree [13]. Concrete software products emerge from the assembly of these features. Moreover, some features depend on others for their inclusion, and the presence of certain features may preclude the selection of others. This

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Q-SE 2024, April 16, 2024, Lisbon, Portugal

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0570-0/24/04...\$15.00
<https://doi.org/10.1145/3643667.3648224>

paper preliminary proposes a FM to manage the inherent variability during the development of hybrid software systems.

The remainder of this paper is structured as follows. Section 2 summarizes the main concepts related to the proposal. Sections 3 and 4 present the proposal and the main results, respectively. Finally, Section 5 presents the conclusions and further research.

2 BACKGROUND

An SPL comprises a suite of similar software products developed from reusable artifacts within a specific application domain [6]. These products share standard features to meet the needs of a particular market segment [12]. The goal of SPLs is to create a family of products based on shared elements while also providing mechanisms to manage the unique aspects of each product [22]. The SPL development approach offers several benefits, including reduced development costs, improved quality, customization, and shorter development times, as noted by various authors [5]. For successful SPL implementation, special attention must be given to aspects such as the planned reuse of components, which are not typically present in other software development methods [2].

Variability is a crucial characteristic of SPLs, providing flexibility for product diversification and differentiation [10]. It refers to an SPL's capacity for change, customization, configuration, or extension in specific contexts. This variability is introduced through the definition of reusable artifacts, such as architectures or components, that are part of a product family's definition. Including or excluding these artifacts in each final product enables the creation of diverse products [22]. Proper variability management is crucial for maximizing return on investment in constructing and maintaining products within an SPL. However, managing this variability is complex, with the primary challenge being the relationship between each SPL's different "features" [11]. Various authors have proposed models to manage SPL variability, most notably based on the FODA method [18], which uses features and relationships depicted in a tree-like graphical form.

An FM is a tree-like structure whose root node represents the application product family. The SPL features are organized along the tree. The leaves of this FM represent individual features or groups of features, which can be assembled to give rise to particular software products [13].

3 HYBRID SOFTWARE PRODUCT LINES

This section explores the creation of hybrid software systems that integrate quantum computing elements with classical software. As QC becomes increasingly influential, adapting and enhancing classical software with quantum functionalities is essential. We present an innovative model to facilitate this combination, offering a new perspective on software development in the quantum era. This model lays the foundation for an approach that enhances traditional SE practices, setting the stage for future advancements in the field. Section 3.1 outlines the overall process and the application of the proposed FM, while Section 3.2 details the FM and discusses its use in expressing variability in quantum-classical software.

3.1 Application process of Hybrid Software Product Lines

A proposed FM for hybrid software systems (FM-H) can be applied together with other domain-specific FMs (FM-D). The FM-H is adaptable to different domains through the FM-D, allowing domain experts to use existing FM-Ds or define new ones. Various FM-Ds exist across multiple domains [4]. FM-D is computing-independent, defining potential software features relevant to an SPL within a specific domain. At this high abstraction level, domain experts select the anticipated features that the possible software variants in the product line might or should support. This selection process forms the application configuration model (CM-D). The CM-D can be further customized with specific constraints (CC - Constraints Configuration) between different features in various configurations.

The second level in this process is performed by the application designers, who obviously should also know about the domain. This level is still a platform-independent level. Application designers will directly use our proposed FM-H, and will select (similarly to the previous level) the set of features associated with the development of hybrid software. This subset of features are integrated in the hybrid software configuration model (CM-H) and are equally constrained in both levels. First, regarding some QC aspects (e.g., *"if the feature 'gate-based' is selected, then 'quantum-Lang' should be 'Qiskit'"*). Second, the CC model could include constraints between two levels (e.g., *"if 'feature 1' is selected, then it could be supported by 'quantum', including both features, 'gate-based' or 'annealing' features."*), indicating that 'feature 1' can be only supported by quantum software.

Finally, quantum and classical software developers create software variants by considering CM-D and CM-H, along with the constraints defined in CC. At this stage, developers can use existing toolchains in the SPL field to produce and evolve these variants.

3.2 Feature Model for Hybrid Software

Since FMs have traditionally been used in SPL to represent and manage variabilities and commonalities in product families [1, 21]. Then, it is crucial to consider how quantum features can be modeled, configured and adapted when integrating QC with SPL. Managing variability in the context of quantum software using FM involves modeling, configuring, and adapting Q-features, considering quantum software relationships, constraints, and particularities. This integration promises to develop more modular, reusable, and adaptable quantum software but also presents unique challenges that must be considered. An initial discussion follows regarding the main sections, points of variation, and features identified.

The approach can be defined following a Feature Modelling perspective in which the SPL approach combines MDE and Requirement Engineering principles. This perspective attempts to use SPL/MDE for managing variability in a technology-independent way. In this approach, it could consider: (i) CIM as domain-specific features and variability constraints; (ii) PIM for defining functionality (classical-quantum, gate based model -annealing), and quantum algorithms; and (iii) PSM as the variability (manually) defined/modeled based on SPL principles, and managed in a (semi) automated way (according to specific constraints/preferences).

According to those above, the variability presents, in a hybrid software system, an initial set of features related to the functionality, e.g., the set of quantum algorithms, the programming languages, the integration mechanisms, and the hardware were considered. Then, each of these features was reviewed in detail, and a subset of features and relationships between them were identified. The details of this model are presented in Figure 1.

FMs offer various modeling concepts, such as optional and mandatory features, a feature hierarchy, feature groups, and cross-tree constraints, as illustrated in Figure 1. Our proposal, called *Hybrid Quantum-Classical SPL (HQC-SPL)*, exhibits five main features, two required (*functionality and programming languages*), and three optionals (*quantum algorithms, integration mechanisms, and hardware-based constraints*). All the rest of the child features are related by an OR relation with their parent feature. The exception is the case of the *HW-based-constraint* feature, where the child features represent one of the three possible levels of hardware constraint.

The *functionality* feature represents a hybrid system, considering a software system's quantum and classical elements. The feature *quantumAlgorithm* describes the diverse options to implement quantum algorithms in a particular solution. The feature *programmingLang*, includes the possibilities for classical or quantum programming languages. The integration mechanisms consider the feature *integrationMech*. The child features *micro-Ss* or *quantumSOA* represent a potential integration using micro-services or quantum SOA services.

In addition, cross-tree constraints are allowed. These constraints consider two cases. The «requires» relationship between *feature A* and *feature B* means that *feature A*'s selection implies *feature B*'s selection. The «excludes» relationships between *feature A* and *feature B* means that *feature A* and *feature B* cannot be part of the same product. For this proposal, selecting the feature *quantum* implies that the *HW-based-constraint* must be selected. Details about the constraints are presented in the lower right corner of Figure 1.

4 DISCUSSION

This section delves into the implications and insights from HQC-SPL proposal for developing hybrid quantum-classical software. It aims to provide a thorough understanding of this innovative method's benefits and challenges. Section 4.1 discusses our proposal's overarching purpose and potential advantages, emphasizing its role in facilitating significant progress in developing hybrid software systems. Section 4.2 outlines the key lessons and practical strategies for researchers and practitioners, highlighting the crucial insights gained from our research. Lastly, Section 4.3 addresses the limitations of our current approach.

4.1 Purpose and Gains of the Proposal

The proposal introduces an innovative method for managing the complexity caused by the combination of quantum and classical parts in hybrid systems, improving their construction and reusability and paving the way for advanced solutions to complex computational problems. It emphasizes enhancing development efficiency and adaptability through applying SPLs principles. This approach is poised to revolutionize the field by seamlessly blending the capabilities of classical and quantum software.

4.2 Lessons learned

For researchers, the proposal highlights the importance of interdisciplinary collaboration between SE and QC. It emphasizes the unique challenges of adapting classical SE methodologies to quantum and hybrid software systems. It reveals SPL's potential in new domains, managing variability and customizing software in the quantum context.

For practitioners, the proposal provides a practical framework for tackling the complexity of developing hybrid quantum-classical software. It underscores the importance of modularity and reuse as critical strategies for enhancing efficiency and scalability in developing hybrid software systems.

This approach can also be integrated with other methods, such as Quantum DevOps [17], being beneficial for managing variability and its impact on operational activities (e.g. selecting quantum devices, understanding how to test and execute specific quantum algorithms).

4.3 Limitations

A significant limitation of the proposal is its potential dependency on specific tools and platforms, by limiting its adaptability and applicability across different technological contexts. The success and efficiency of the framework are greatly influenced by the chosen tools and platforms, which could hinder its broad adoption and implementation.

Implementing the proposal requires specialized technical expertise in QC, which is currently unavailable. This poses a significant obstacle to developing quantum-classical software, slowing down the adoption and refinement of the proposed methodologies and principles.

5 CONCLUSIONS AND FUTURE DIRECTIONS

This study presents a preliminary application of SPL principles to the domain of QC for managing the variability in hybrid (classical/quantum) software systems. The proposed approach successfully uses FMs to represent and handle this variability, allowing for a clear delineation of commonalities and differences across a spectrum of software products. This structured approach to variability management is crucial in addressing the unique challenges of integrating quantum algorithms into classical computing environments. The use of FMs not only facilitates the customization and scalability of hybrid systems but also provides a systematic way to capture the evolving landscape of quantum computing capabilities. The conclusions underscore the importance of robust variability management in realizing the full potential of quantum-classical software development methods, setting the stage for future research to expand upon this initial approach to accommodate the rapid evolution in quantum software technologies.

We will focus on expanding the scope of the SPL framework to encompass a broader range of QC scenarios and to refine the granularity of FMs. We aim to develop tool support for automating the configuration and deployment of hybrid software systems, addressing the dynamic nature of quantum software development.

The future work considers developing specific tools and frameworks to enhance the application of SPL in hybrid software systems. Additionally, there is a focus on conducting empirical validations

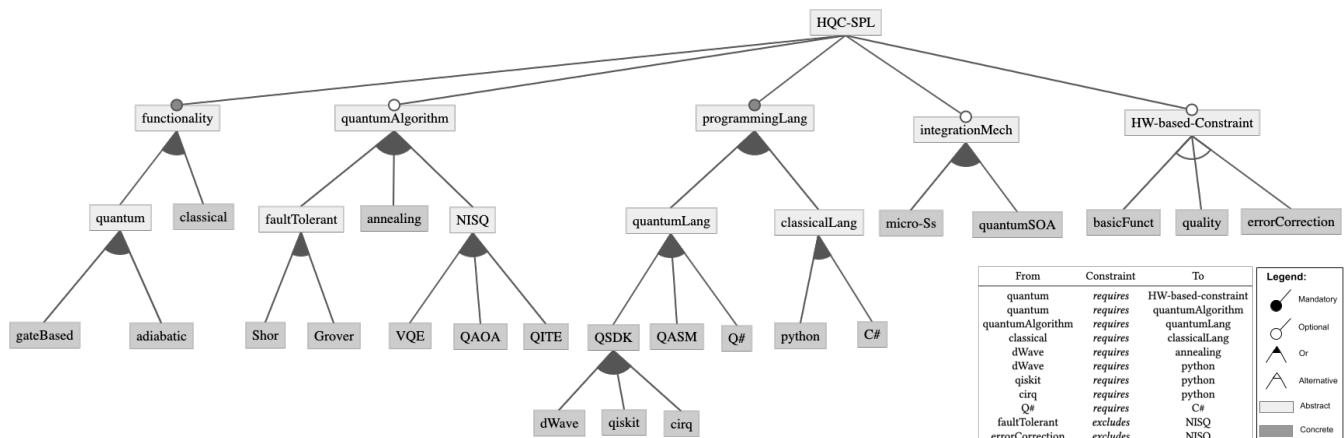


Figure 1: FM for the HQC-SPL proposal.

and case studies to assess the proposal's efficacy in various scenarios. Furthermore, the proposal aims to explore new application areas, investigating how it can be adapted and applied in diverse domains like cryptography, process optimization, and complex system simulation. This approach underscores the commitment to expanding the scope and impact of the proposed variability identification on quantum and classical software.

ACKNOWLEDGMENTS

Samuel Sepúlveda thanks to University of La Frontera, Vicerrectoría de Investigación y Postgrado, project GI23-0012. Pérez and Piattini would thank the grants PID2022-137944NB-I00 (SMOOTH Project) and PDC2022-133051-I00 (QU-ASAP Project) funded by MCIN/AEI/ 10.13039/501100011033 and by the “European Union NextGenerationEU/PRTR”

REFERENCES

- and case studies to assess the proposal's efficacy in various scenarios. Furthermore, the proposal aims to explore new application areas, investigating how it can be adapted and applied in diverse domains like cryptography, process optimization, and complex system simulation. This approach underscores the commitment to expanding the scope and impact of the proposed variability identification on quantum and classical software.
- ## ACKNOWLEDGMENTS
- Samuel Sepúlveda thanks to University of La Frontera, Vicerrectoría de Investigación y Postgrado, project GI23-0012. Pérez and Piattini would thank the grants PID2022-137944NB-I00 (SMOOTH Project) and PDC2022-133051-I00 (QU-ASAP Project) funded by MCIN/AEI/ 10.13039/501100011033 and by the “European Union NextGenerationEU/PRTR”
- ## REFERENCES
- [1] O. Aguayo and S. Sepúlveda. 2022. Variability management in dynamic software product lines for self-adaptive systems—a systematic mapping. *Applied Sciences* 12, 20 (2022), 10240.
 - [2] F. Ahmed and L. F. Capretz. 2008. Best practices of RUP® in software product line development. In *2008 Int. Conf. on Computer and Communication Engineering*. IEEE, Kuala Lumpur, Malaysia, 1363–1366.
 - [3] Y. Alexeev, A. McCaskey, and W. De Jong. 2022. Introduction to the special issue on software tools for quantum computing: Part 1. *ACM Transactions on Quantum Computing* 3, 3 (2022), 1–3.
 - [4] D. Amalfitano, V. De Simone, A. Fasolino, M. Lubrano, and S. Scala. 2016. Introducing Software Product Lines in Model-Based Design Processes: An Industrial Experience. In *2016 13th Working IEEE/IFIP Conf. on Software Architecture (WICSA)*. IEEE, Venice, Italy, 287–290.
 - [5] S. Apel, D. Batory, C. Kästner, and G. Saake. 2016. *Feature-oriented software product lines*. Springer-Verlag Berlin An, Berlin, Germany.
 - [6] H. Arboleda and J. C. Royer. 2012. *Model-driven and software product line engineering*. ISTE Limited, London, England.
 - [7] L.S. Barbosa. 2020. Software engineering for ‘quantum advantage’. In *Procs. of the IEEE/ACM 42nd Int. Conf. on Software Engineering Workshops*. IEEE, ACM, New York, United States, 427–429.
 - [8] J. Bosch, G. Florijn, D. Greefhorst, J. Kuusela, J. H. Obbink, and K. Pohl. 2001. Variability issues in software product lines. In *International Workshop on Software Product-Family Engineering*. Springer, Berlin, Heidelberg, Bilbao, Spain, 13–21.
 - [9] A. Carleton, J.E. Harper, E. Robert, M.H. Klein, E. De Niz, D. Desautels, J.B. Goodenough, C. Holland, I. Ozkaya, D. Schmidt, F.J. Shull, J. Herbsleb, J. Foreman, and R. Cunningham. 2021. *Architecting the future of software engineering: A national agenda for software engineering research and development*. Technical Report. Carnegie-Mellon Univ., SEI.
 - [10] L. Chen and M.A. Babar. 2011. A systematic review of evaluation of variability management approaches in software product lines. *Information and Software Technology* 53, 4 (2011), 344–362.
 - [11] A. Classen, P. Heymans, and P. Y. Schobbens. 2008. What’s in a feature: A requirements engineering perspective. In *Int. Conf. on Fundamental Approaches to Software Engineering*. Springer, Berlin, Germany, 16–30.
 - [12] P. C. Clements and L. Northrop. 2001. *Software Product Lines: Practices and Patterns*. Addison-Wesley, NJ, USA.
 - [13] K. Czarnecki and A. Wasowski. 2007. Feature diagrams and logics: There and back again. In *11th Int. Software Product Line Conference (SPLC)*. IEEE, Kyoto, Japan, 23–34.
 - [14] M. De Stefano, F. Pecorelli, D. Di Nucci, F. Palomba, and A. De Lucia. 2022. Software engineering for quantum programming: How far are we? *Journal of Systems and Software* 190 (2022), 1–15.
 - [15] M. Felderer, D. Taibi, F. Palomba, M. Epping, M. Lochau, and B. Weder. 2023. Software Engineering Challenges for Quantum Computing: Report from the First Working Seminar on Quantum Software Engineering (WSQE 22). *ACM SIGSOFT Software Engineering Notes* 48, 2 (2023), 29–32.
 - [16] M. Galster, D. Weyns, D. Tofan, B. Michalik, and P. Avgeriou. 2014. Variability in software systems—a systematic literature review. *IEEE Transactions on Software Engineering* 40, 3 (2014), 282–306.
 - [17] I.D. Gheorghe-Pop, N. Tcholtchev, T. Ritter, and M. Hauswirth. 2020. Quantum DevOps: Towards Reliable and Applicable NISQ Quantum Computing. In *2020 IEEE Globecom Workshops (GC Wkshps)*. IEEE, Taipei, Taiwan, 1–6.
 - [18] K. C. Kang, S. G. Cohen, J. A. Hess, W. E. Novak, and A. S. Peterson. 1990. *Feature-oriented domain analysis (FODA) feasibility study*. Technical Report CMU/SEI-90-TR-21. Carnegie-Mellon University, SEI.
 - [19] F. Leymann and J. Barzen. 2021. Hybrid Quantum Applications Need Two Orchestration in Superposition: A Software Architecture Perspective. *arXiv:2103.04320*
 - [20] M. A. Serrano, R. Pérez-Castillo, and M. Piattini. 2022. *Quantum Software Engineering*. Springer, Switzerland.
 - [21] M. Setyautami, H.S. Fadhilillah, D. Adianto, I. Affan, and A. Azurat. 2020. Variability management: Re-engineering microservices with delta-oriented software product lines. In *Procs. of the 24th ACM Conf. on Systems and Software Product Line: Volume A*. ACM, Montreal Quebec Canada, 1–6.
 - [22] M. Sinnema and S. Deelstra. 2008. Industrial validation of COVAMOF. *Journal of Systems and Software* 81, 4 (2008), 584–600.
 - [23] B. Weder, J. Barzen, F. Leymann, and D. Vietz. 2022. *Quantum Software Development Lifecycle*. Springer International Publishing, Cham, 61–83.
 - [24] J. Zhao. 2020. Quantum software engineering: Landscapes and horizons. *arXiv preprint arXiv:2007.07047* 1, 1 (2020), 1–34.